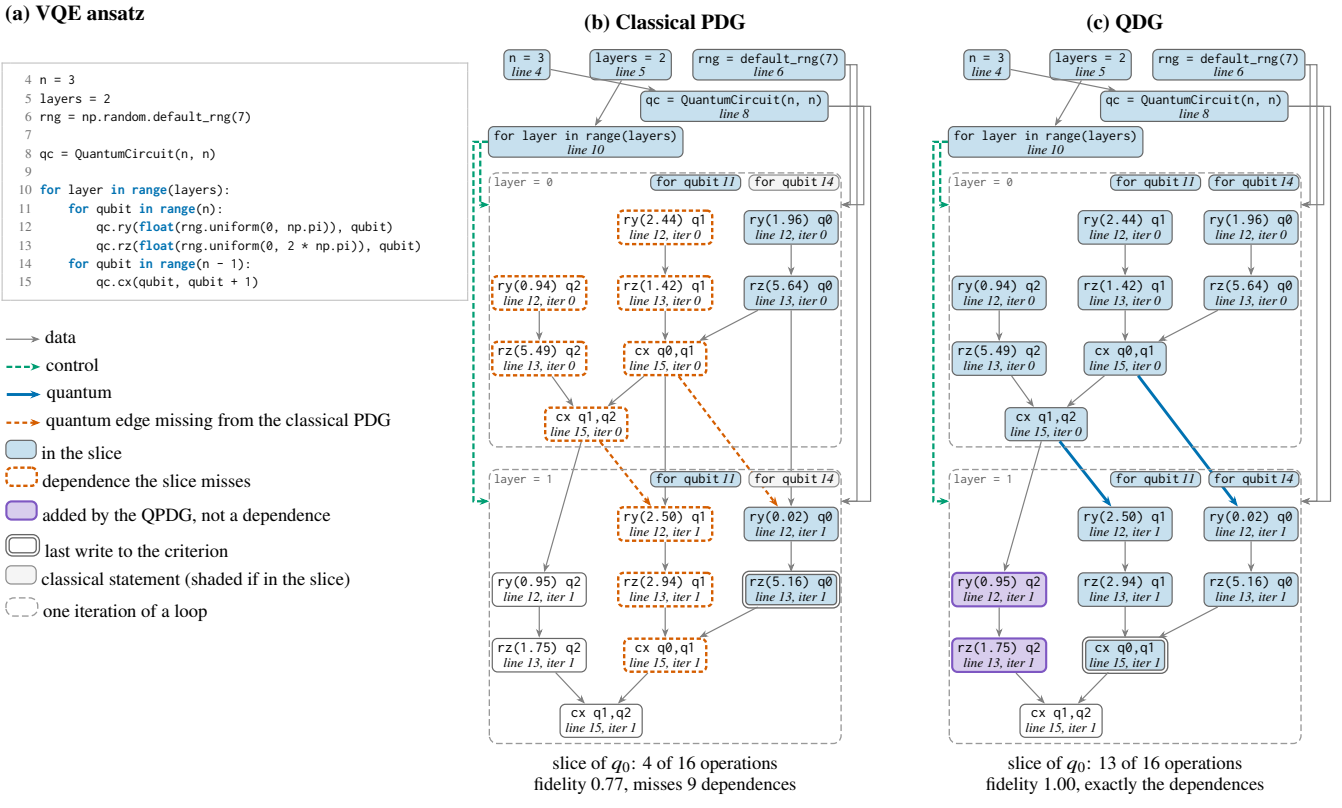

(a) VQE ansatz
4 n = 3
5 layers = 2
6 rng = np.random.default_rng(7)
7
8 qc = QuantumCircuit(n, n)
9
10 for layer in range(layers):
11 for qubit in range(n):
12 qc.ry(float(rng.uniform(0, np.pi)), qubit)
13 qc.rz(float(rng.uniform(0, 2 * np.pi)), qubit)
14 for qubit in range(n - 1):
15 qc.cx(qubit, qubit + 1)
data
control
quantum
quantum edge missing from the classical PDG
in the slice
dependence the slice misses
added by the QPDG, not a dependence
last write to the criterion
classical statement (shaded if in the slice)
one iteration of a loop
(b) Classical PDG
n = 3 line 4
layers = 2 line 5
rng = default_rng(7) line 6
qc = QuantumCircuit(n, n) line 8
for layer in range(layers) line 10
layer = 0
for qubit 11
for qubit 14
ry(2.44) q1 line 12, iter 0
ry(1.96) q0 line 12, iter 0
ry(0.94) q2 line 12, iter 0
rz(1.42) q1 line 13, iter 0
rz(5.64) q0 line 13, iter 0
rz(5.49) q2 line 13, iter 0
cx q0,q1 line 15, iter 0
cx q1,q2 line 15, iter 0
layer = 1
for qubit 11
for qubit 14
ry(2.50) q1 line 12, iter 1
ry(0.02) q0 line 12, iter 1
ry(0.95) q2 line 12, iter 1
rz(2.94) q1 line 13, iter 1
rz(5.16) q0 line 13, iter 1
rz(1.75) q2 line 13, iter 1
cx q0,q1 line 15, iter 1
cx q1,q2 line 15, iter 1
slice of q_0: 4 of 16 operations
fidelity 0.77, misses 9 dependences
(c) QDG
n = 3 line 4
layers = 2 line 5
rng = default_rng(7) line 6
qc = QuantumCircuit(n, n) line 8
for layer in range(layers) line 10
layer = 0
for qubit 11
for qubit 14
ry(2.44) q1 line 12, iter 0
ry(1.96) q0 line 12, iter 0
ry(0.94) q2 line 12, iter 0
rz(1.42) q1 line 13, iter 0
rz(5.64) q0 line 13, iter 0
rz(5.49) q2 line 13, iter 0
cx q0,q1 line 15, iter 0
cx q1,q2 line 15, iter 0
layer = 1
for qubit 11
for qubit 14
ry(2.50) q1 line 12, iter 1
ry(0.02) q0 line 12, iter 1
ry(0.95) q2 line 12, iter 1
rz(2.94) q1 line 13, iter 1
rz(5.16) q0 line 13, iter 1
rz(1.75) q2 line 13, iter 1
cx q0,q1 line 15, iter 1
cx q1,q2 line 15, iter 1
slice of q_0: 13 of 16 operations
fidelity 1.00, exactly the dependences